

VASP sur GPU

Ce qui accélère un calcul, ce qui ne l'accélère pas,
et comment utiliser au mieux les deux H100 de vm1-ic2mp

Gilles Frapper

Professeur des Universités, Chimie Théorique
IC2MP UMR 7285 CNRS, Université de Poitiers
gilles.frapper@univ-poitiers.fr

2 octobre 2026

Résumé

Ce rapport rassemble, sous forme pratique, les enseignements des campagnes de mesure menées du 26 septembre au 2 octobre 2026 avec VASP 6.5.1 (version GPU OpenACC) sur le nœud vm1-ic2mp (2 GE NVIDIA H100 NVL). Plus de 350 calculs ont été analysés, sans compter les phonons : points simples et optimisations PBE et r²SCAN de 1 à 64 atomes sur 1 et 2 GPU, phonons, calculs simultanés sur une GPU avec et sans MPS, dynamique moléculaire de LiN₃ (144 atomes), et deux campagnes consacrées au paramètre NSIM jusqu'à 216 atomes, en PBE et HSE06. Le message principal est simple : jusqu'à une centaine d'atomes, un calcul VASP n'occupe qu'une petite partie d'une H100. Les leviers efficaces ne sont pas les réglages fins de l'INCAR (NSIM n'apporte rien : $\pm 5\%$), mais l'organisation des calculs : un calcul par GPU plutôt qu'un calcul sur deux GPU, et plusieurs calculs sur la même GPU sous MPS, ce qui multiplie le débit par 2,5 à 5,8.

L'essentiel en une page

Question	Réponse mesurée sur vm1-ic2mp
Un calcul de 1 à 64 atomes remplit-il une H100 ?	Non. Utilisation moyenne de 10 à 50 %, mémoire de 2,8 à 12,2 Go sur 94. Vers 150–200 atomes, l'utilisation monte à 64–75 %.
Faut-il deux GPU pour un calcul ?	Rarement. Gain de 1,0 à 1,9× seulement. Deux calculs indépendants, un par GPU, donnent plus de débit.
Sur deux GPU, KPAR = 1 ou 2 ?	KPAR = 2 dès qu'il y a plus de 4 points k (42 cas sur 56) ; KPAR = 1 seulement avec 3–4 points k et des mailles de 64 atomes.
Faut-il augmenter NSIM ?	Non. De 4 à 128 : au mieux +6 %, souvent rien, et jusqu'à –5 % avec +27 Go de mémoire pour Si 216 atomes à NSIM = 128. Garder la valeur par défaut ou 8–32.
Plusieurs petits calculs à la fois ?	Oui, mais uniquement sous MPS : débit $\times 4$ à 5,8 avec 8 calculs par GPU. Sans MPS, le lot est jusqu'à 1,8 fois <i>plus lent</i> que les calculs en série.
vasp_gam pour le point Γ ?	Gain modeste : 10 % de temps et 27 % de mémoire GPU en moins (AIMD LiN ₃ , 144 atomes).
Combien coûtent r ² SCAN et HSE06 ?	r ² SCAN : 1,6 à 8× PBE en point simple, jusqu'à 16× en optimisation. HSE06 : environ 60× PBE par pas électronique (MgO, 32 atomes).
Les résultats sont-ils fiables ?	Oui. Temps reproductibles à 1 % ; énergies identiques sur 1 et 2 GPU (0,01 meV/atome) et quel que soit NSIM.

Table des matières

L'essentiel en une page	1
1 Contexte	2
1.1 Machine et logiciel	2
1.2 Comment VASP utilise la GPU	2
1.3 Campagnes utilisées	3
2 Un petit calcul n'occupe qu'une partie de la GPU	3
2.1 Temps de calcul	3
2.2 Utilisation et mémoire	4
3 Coût des méthodes	4
3.1 r^2 SCAN par rapport à PBE	4
3.2 HSE06	4
3.3 Optimisation et point simple	5
4 Réglages de parallélisation	5
4.1 Une ou deux GPU, et KPAR	5
4.2 NSIM : aucun effet notable	6
4.3 vasp_gam au point Γ	7
5 Plusieurs calculs sur une même GPU : MPS	8
6 Ordres de grandeur pour quelques flux de travail	9
7 Fiabilité des résultats	9
8 Recommandations pratiques	10
9 Limites et suite	10

1 Contexte

1.1 Machine et logiciel

GPU	2 GE NVIDIA H100 NVL, 94 Go chacune (pilote 595.84)
CPU	AMD EPYC 7662, 32 cœurs attribués à la VM ; 62 Go de mémoire
Ressources Slurm par GPU	16 cœurs, ~28 Go de RAM, pas de limite de temps
VASP	6.5.1, version GPU OpenACC (<code>vasp_std</code> , <code>vasp_gam</code>), compilée le 23/09/2026
Chaîne de compilation	NVHPC 26.3, OpenMPI HPC-X, Intel MKL
Exécution	1 rang MPI par GPU ; <code>OMP_NUM_THREADS</code> = 1 (8 pour le scan de NSIM)

La machine est partagée entre plusieurs utilisateurs : sauf mention contraire, un seul calcul de l'étude tournait à la fois. Certaines mesures ont toutefois été faites alors que d'autres jobs occupaient le nœud ; c'est signalé à chaque fois, car cela peut fausser des écarts de quelques pour cent.

1.2 Comment VASP utilise la GPU

Dans la version OpenACC de VASP, presque tout le travail numérique (transformées de Fourier, algèbre linéaire dense sur les bandes, application de l'hamiltonien) s'exécute sur la GPU ; le CPU orchestre. Quelques conséquences pratiques :

- on lance **un rang MPI par GPU** ; ajouter des rangs n'accélère rien et des threads OpenMP n'aident que les parties restées sur le CPU ;
- **NCORE** est imposé à 1 par le code ; la parallélisation entre GPU se règle avec **KPAR** (répartition des points k) ;
- **NSIM** fixe le nombre de bandes traitées ensemble dans les algorithmes itératifs : des blocs plus gros forment de plus grosses opérations matricielles, ce qui, en principe, convient mieux à une GPU ;
- une GPU n'est efficace que si chaque opération est assez grosse pour occuper ses milliers de cœurs. Pour un petit système, le temps est dominé par les latences (lancement des noyaux, synchronisations, transferts hôte-GPU), pas par le calcul.

1.3 Campagnes utilisées

Campagne	Contenu	Calculs
Séries d'échelle (phases 1–4)	Si, Al, MgO de 1 à 64 atomes ; PBE et r ² SCAN (jeux Materials Project) ; point simple et optimisation ; 1 GPU, 2 GPU avec KPAR = 1 et 2	186
Phonons (phase 5)	Si, Al, MgO par déplacements finis (phonopy), 1 et 2 GPU	6 chaînes
Calculs simultanés (phase 6)	1 à 8 copies d'un même calcul sur une GPU, avec et sans MPS	100
AIMD (phase 7, calibration)	LiN ₃ , 144 atomes, PBE+D3, 200 pas, vasp_gam et vasp_std	2
test2	les 56 calculs 1 GPU des séries d'échelle refaits avec NSIM = 32	56 + 3 chaînes
Scan NSIM	NSIM = 4 à 128 ; MgO 32, LiN ₃ 144 et Si 216 atomes en PBE ; MgO 32 en HSE06	24

Les paramètres physiques (**ENCUT**, points k, critères de convergence) sont ceux de Materials Project, générés par pymatgen ; ils sont détaillés dans l'annexe du rapport principal (**report/report.tex**, en anglais).

2 Un petit calcul n'occupe qu'une partie de la GPU

2.1 Temps de calcul

Les temps absolus sont courts (tableau 1) : tous les calculs PBE de 1 à 64 atomes durent moins de 5,5 min ; le plus long, l'optimisation r²SCAN de MgO à 64 atomes, 48 min.

TABLE 1 – Temps total (s) sur 1 GPU, séries d'échelle. SP : point simple ; relax : optimisation complète (ISIF = 3).

	Atomes	PBE SP	PBE relax	r ² SCAN SP	r ² SCAN relax
Si	2	15	20	24	31
Si	16	40	69	101	215
Si	64	110	175	324	693
Al	1	14	14	39	101
Al	32	34	57	181	594
Al	64	83	122	654	1943
MgO	2	36	46	31	66
MgO	16	97	127	173	696
MgO	64	163	309	470	2864

Le temps par pas SCF croît *moins vite* que le nombre d’atomes : entre 8 et 64 atomes, $t_{\text{SCF}} \propto N^{0,6-0,95}$ (figure 1). Cela tient en partie à la méthodologie MP, qui fixe une densité de points k : une maille plus grande a moins de points k . Mais même par point k , l’exposant (1,25–1,4) reste très loin du N^2 – N^3 attendu lorsque la GPU est saturée. Pour les plus petites mailles, le temps par pas est presque constant (environ 1 s en PBE pour Si et Al jusqu’à 8 atomes) : ce sont les coûts fixes qui dominent.

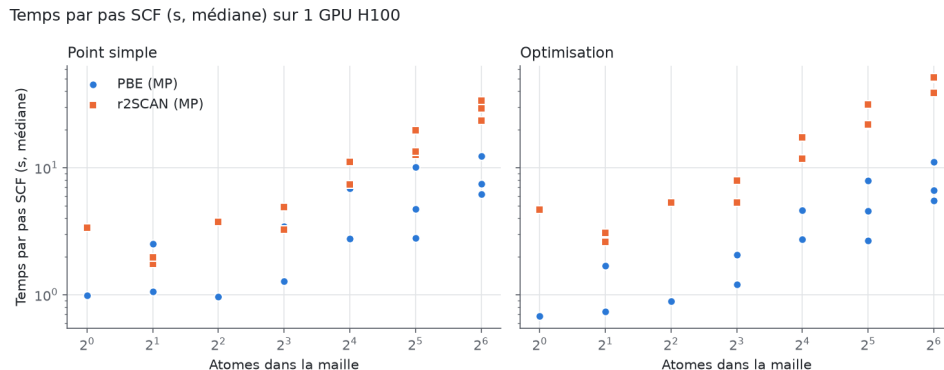


FIGURE 1 – Temps médian par pas SCF en fonction du nombre d’atomes, 1 GPU.

2.2 Utilisation et mémoire

- **Utilisation moyenne** (fraction du temps où au moins un noyau tourne, relevée toutes les 2s par `nvidia-smi`) : 10 à 50 % de 1 à 64 atomes. Cette mesure surestime l’occupation réelle : un noyau qui n’utilise qu’une partie des multiprocesseurs compte comme « GPU occupée ».
- **Mémoire GPU** : environ 2,9 Go dès un atome (contexte CUDA, bibliothèques, tableaux fixes de VASP), puis au plus 5,2 Go en PBE et 12,2 Go en r²SCAN (Al, 64 atomes, 12 points k). La mémoire n’est jamais une contrainte dans cette gamme.
- **Systèmes plus gros** : dans le scan de NSIM, l’utilisation atteint 64 % pour Si 216 atomes et 75 % pour LiN₃ 144 atomes (8 points k), contre 50 % pour MgO 32 atomes. La GPU commence à être bien employée vers 150–200 atomes.

Ce que cela veut dire. Ces chiffres ne montrent pas que les H100 conviennent mal à VASP ; ils montrent que des mailles de quelques dizaines d’atomes sont trop petites pour elles. Toute la suite en découle : pour gagner du temps sur de petits systèmes, il faut surtout *remplir* la GPU, pas chercher à accélérer un calcul isolé.

3 Coût des méthodes

3.1 r²SCAN par rapport à PBE

Le tableau 2 compare les deux méthodologies MP telles qu’on les utilise : r²SCAN cumule une fonctionnelle plus chère, une énergie de coupure plus haute (680 contre 520 eV) et parfois plus de points k (Al 64 atomes : 12 contre 3).

L’écart grandit avec la taille. En optimisation, r²SCAN demande en plus davantage de pas ioniques (1 à 4) et SCF (13 à 46), car la maille s’ajuste à la nouvelle fonctionnelle à partir de structures relaxées en PBE.

3.2 HSE06

Sur MgO 32 atomes (144 bandes, 12 points k), avec les mêmes réglages que le calcul PBE du scan de NSIM, un pas électronique hybride HSE06 prend 481 à 487 s, contre 7,7 s en PBE : **environ**

TABLE 2 – Rapport des temps r^2 SCAN / PBE sur 1 GPU, du plus petit au plus grand système de chaque série.

	Point simple		Optimisation	
	total	par pas SCF	total	par pas SCF
Si (2 $\rightarrow$ 64 at.)	1,6 $\rightarrow$ 3,0	1,7 $\rightarrow$ 3,2	1,6 $\rightarrow$ 4,0	3,5 $\rightarrow$ 5,8
Al (1 $\rightarrow$ 64 at.)	2,8 $\rightarrow$ 7,9	3,4 $\rightarrow$ 5,5	7,1 $\rightarrow$ 15,9	6,9 $\rightarrow$ 9,3
MgO (2 $\rightarrow$ 64 at.)	0,9 $\rightarrow$ 2,9	0,8 $\rightarrow$ 2,4	1,5 $\rightarrow$ 9,3	1,8 $\rightarrow$ 4,5

60 fois plus. Un point simple HSE06 de 32 atomes représente donc plusieurs heures de GPU ; c’est le domaine où le choix du nombre de points k et des réductions de grille (NKRED) pèse le plus, bien plus que tout réglage de parallélisation.

3.3 Optimisation et point simple

Les structures de départ étant déjà relaxées en PBE par MP, une optimisation PBE coûte 0,9 à 1,9 fois un point simple (1 à 3 pas ioniques). En r^2 SCAN, le rapport monte à 1,3–6,1, le plus élevé pour MgO 64 atomes.

4 Réglages de parallélisation

4.1 Une ou deux GPU, et KPAR

Les 56 calculs des séries d’échelle ont été refaits sur 2 GPU, avec les bandes réparties (KPAR = 1) ou les points k répartis (KPAR = 2). Les énergies sont identiques à celles sur 1 GPU (écart maximal 0,01 meV/atome).

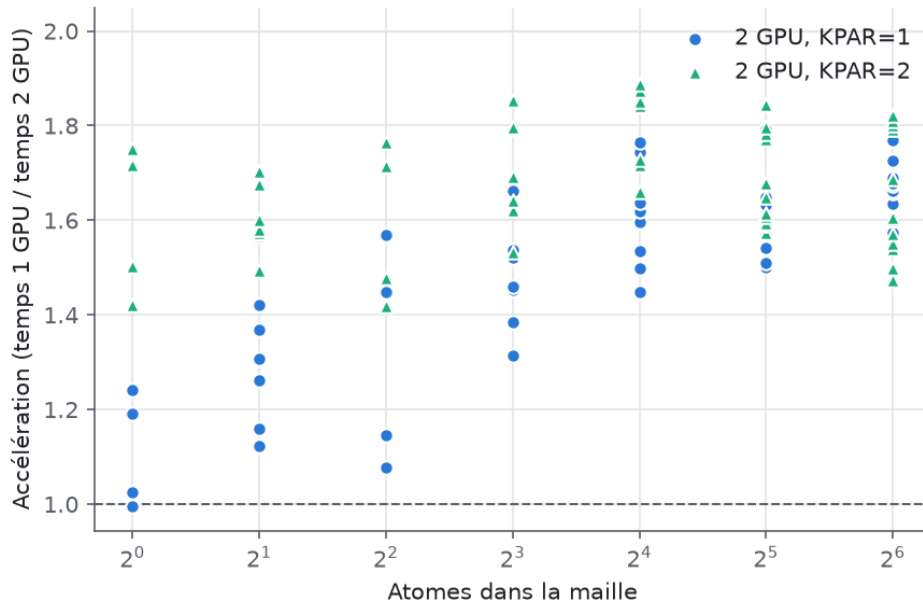


FIGURE 2 – Accélération sur 2 GPU (temps sur 1 GPU / temps sur 2 GPU).

- **L’accélération est modeste** : 0,99 à 1,79 $\times$ avec KPAR = 1, 1,42 à 1,89 $\times$ avec KPAR = 2. Deux GPU ne divisent jamais le temps par deux, alors que deux calculs indépendants, un par GPU, le font par construction.

- **KPAR = 2 est le bon choix par défaut** : plus rapide dans 42 cas sur 56, notamment pour les petites mailles à nombreux points k (10 à 84), où répartir les bandes ne rapporte presque rien (Al 1 atome : $1,0\times$ avec KPAR = 1).
- **KPAR = 1 ne gagne qu'à 64 atomes** avec 3 ou 4 points k (6 cas), en particulier quand les points k se répartissent mal (3 points : $2 + 1$). Égalité à 2 % près dans les 8 cas restants.

Pour les phonons, le temps VASP d'une chaîne complète passe de 0,5–1,7 h sur 1 GPU à un temps 1,4 à 1,6 fois plus court sur 2 GPU, mais le coût en heures de GPU augmente de 25 à 45 % (Si : $0,47 \rightarrow 0,58$ GPU · h ; MgO : $1,73 \rightarrow 2,52$ GPU · h). Sur une machine partagée, l'attente d'une deuxième GPU libre a d'ailleurs allongé le temps de restitution : 8,1 h pour la campagne à 2 GPU contre 6,0 h à 1 GPU.

4.2 NSIM : aucun effet notable

NSIM vaut 4 par défaut. On lit souvent qu'il faut l'augmenter sur GPU pour former de plus grosses opérations matricielles. Deux campagnes indépendantes ont testé cette idée.

Scan de NSIM (4 à 128). Travail électronique identique d'une valeur à l'autre (nombre de pas imposé : 20 en PBE, 12 en HSE06), 1 GPU, une exécution par valeur.

TABLE 3 – Temps par pas électronique (s) selon NSIM, 1 GPU. Moyenne des pas après les 3 premiers (4 en HSE06). En gras : valeur la plus rapide. Dernière ligne de chaque bloc : pic de mémoire GPU (Go).

Système		4	8	16	32	64	128
MgO 32 at., PBE (144 bandes, 12 pts k)	s/pas	7,90	7,70	7,68	7,67	7,77	7,68
	Go	4,7	4,8	4,9	5,1	5,5	5,9
LiN ₃ 144 at., PBE (440 bandes, 8 pts k)	s/pas	45,0	43,9	43,5	42,7	42,6	42,8
	Go	10,9	11,1	11,1	13,1	15,9	19,8
Si 216 at., PBE (606 bandes, point Γ)	s/pas	19,8	19,6	19,7	19,7	20,3	20,8
	Go	12,1	12,7	14,7	18,0	24,2	39,3
MgO 32 at., HSE06	s/pas	483,6	481,1	483,2	486,6	484,4	486,5

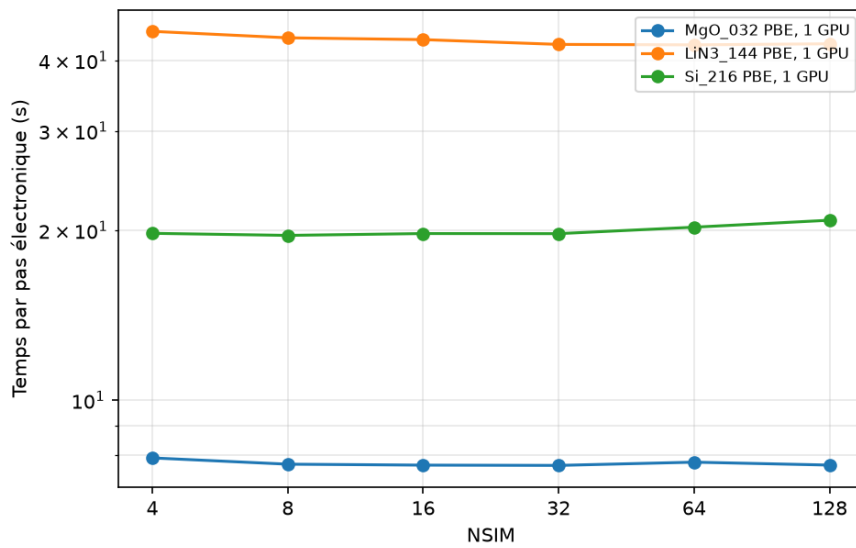


FIGURE 3 – Temps par pas électronique selon NSIM, PBE, 1 GPU (échelle logarithmique).

Le meilleur gain par rapport à $\text{NSIM} = 4$ est de 3 % (MgO PBE), 6 % (LiN_3), 1 % (Si 216) et 0,5 % (HSE06). Pour Si 216 atomes, les grandes valeurs sont même nuisibles : -5% à $\text{NSIM} = 128$, avec une mémoire GPU multipliée par 3,2. Les énergies ne dépendent pas de NSIM (écarts $\leq 4 \times 10^{-5}$ eV par maille, dus à l'arrêt de la SCF après un nombre fixe de pas). Réserve : les 18 calculs PBE ont tourné alors que d'autres jobs occupaient le nœud ; les écarts de quelques pour cent sont du même ordre que cet effet.

Campagne test2 ($\text{NSIM} = 32$ contre 4). Les 56 calculs 1 GPU des séries d'échelle ont été refaits à l'identique, avec $\text{NSIM} = 32$:

- temps par pas SCF : rapport moyen (géométrique) de 1,005 ; médiane par série entre 0,99 et 1,03 ; extrêmes 0,89 et 1,06 ;
- nombre de pas SCF identique dans 53 cas sur 56. Deux optimisations r^2SCAN de MgO (32 et 64 atomes) sont 12 à 16 % plus longues, mais par un chemin de convergence différent (39 $\rightarrow$ 44 pas SCF pour 32 atomes), pas par un pas plus lent ;
- énergies : écart maximal $1,4 \times 10^{-4}$ eV par maille.

Conclusion sur NSIM . Sur H100, de 2 à 216 atomes, en PBE, r^2SCAN et HSE06, NSIM ne modifie pas le temps de plus de quelques pour cent. Il n'y a pas lieu de le régler ; si on le fait, rester entre 8 et 32 et éviter les grandes valeurs au point Γ sur de gros systèmes, qui coûtent de la mémoire sans rien rapporter.

Pourquoi NSIM ne change rien ici. On lit couramment que, sur GPU, la valeur par défaut $\text{NSIM} = 4$ sous-exploite la carte et qu'il faut passer à 16–32, voire plus sur A100/H100. Nos mesures ne le confirment pas, et les OUTCAR permettent de comprendre pourquoi :

1. **La valeur demandée est bien appliquée** ($\text{NSIM} = 128$ dans l'OUTCAR) : l'absence d'effet ne vient pas d'un paramètre ignoré.
2. **La partie concernée domine pourtant le temps.** La mise à jour des orbitales (RMM-DIIS et Davidson, les routines où NSIM intervient) représente 92 à 93 % de la boucle SCF pour LiN_3 , 78 à 79 % pour Si 216 et 99 % en HSE06. Passer de 4 à 64 ne réduit le temps RMM-DIIS de LiN_3 que de 6 % (633 $\rightarrow$ 597 s) ; pour Si 216, passer à 128 l'augmente de 4 %.
3. **Les noyaux sont déjà assez gros à $\text{NSIM} = 4$.** L'argument de l'occupation vaut pour des opérations trop petites. Or une seule transformée de Fourier sur la grille de Si 216 atomes ($126^3 \approx 2 \times 10^6$ points) offre déjà plus de travail parallèle qu'une H100 n'a de fils actifs à la fois (132 multiprocesseurs $\times$ 2048 fils $\approx 2,7 \times 10^5$). Les produits matriciels sur les bandes (orthogonalisation, diagonalisation dans le sous-espace) portent sur toutes les bandes et ne dépendent pas de NSIM . Grouper davantage de bandes n'apporte alors que des opérations plus grosses mais pas plus efficaces, et occupe plus de mémoire.
4. **En HSE06**, le coût est celui de l'échange exact, calculé paire de bandes par paire de bandes et point k par point k ; NSIM n'agit pas sur ce regroupement.

L'utilisation moyenne inférieure à 100 % (50 à 75 % dans ces calculs) traduit plutôt des temps morts entre noyaux (lancements, synchronisations, parties restées sur le CPU) que des noyaux trop petits, ce que NSIM ne peut pas corriger. Cette explication reste à confirmer par un profilage (par exemple Nsight Systems) ; il est possible que NSIM compte davantage pour des systèmes bien plus gros ou sur d'autres GPU. Les recommandations générales trouvées dans la littérature restent valables sur un point, confirmé ici : un NSIM élevé coûte de la mémoire GPU (Si 216 : 12 $\rightarrow$ 39 Go de 4 à 128).

4.3 vasp_gam au point Γ

Pour une AIMD de LiN_3 (144 atomes, point Γ seul, PBE+D3, polarisé en spin), `vasp_gam`, qui exploite la symétrie réelle des fonctions d'onde, prend 61,2 s par pas ionique (13 pas SCF par

pas, utilisation moyenne 37 %, 5,6 Go). Sur les 63 premiers pas, à trajectoire identique, **vasp_std** prend 67,6s par pas et 7,7 Go. Le gain de **vasp_gam** est donc d'environ 10 % en temps et 27 % en mémoire : utile, sans plus. Mesure à confirmer sur 200 pas (le calcul **vasp_std** n'est pas allé au bout).

5 Plusieurs calculs sur une même GPU : MPS

Puisqu'un petit calcul laisse la GPU en grande partie inoccupée, on peut en lancer plusieurs en même temps sur la même carte. Deux modes ont été comparés, avec N copies identiques d'un point simple, dans un seul job Slurm, sans autre job sur le nœud :

- **sans MPS** : chaque processus a son propre contexte CUDA et la GPU passe de l'un à l'autre par tranches de temps ;
- **avec MPS** (*Multi-Process Service* de NVIDIA) : un serveur unique regroupe les processus, dont les noyaux s'exécutent réellement en même temps.

Gain de débit = $N \times t_{\text{ref}} / T_{\text{lot}}$, où t_{ref} est le temps du calcul seul et T_{lot} celui du lot entier (idéal : N).

TABLE 4 – Gain de débit de N calculs simultanés sur une H100.

Série	Composé	At.	t_{ref} (s)	N = 2		N = 4		N = 8	
				sans	MPS	sans	MPS	sans	MPS
PBE SP	Si	8	18	0,56	1,90	0,61	3,47	0,60	5,78
PBE SP	MgO	32	138	0,55	1,89	0,58	3,31	0,56	5,04
r2SCAN SP	Al	32	181	0,93	1,81	1,08	2,94	1,15	4,12
r2SCAN SP	Si	64	324	1,11	1,67	1,35	2,53	–	–

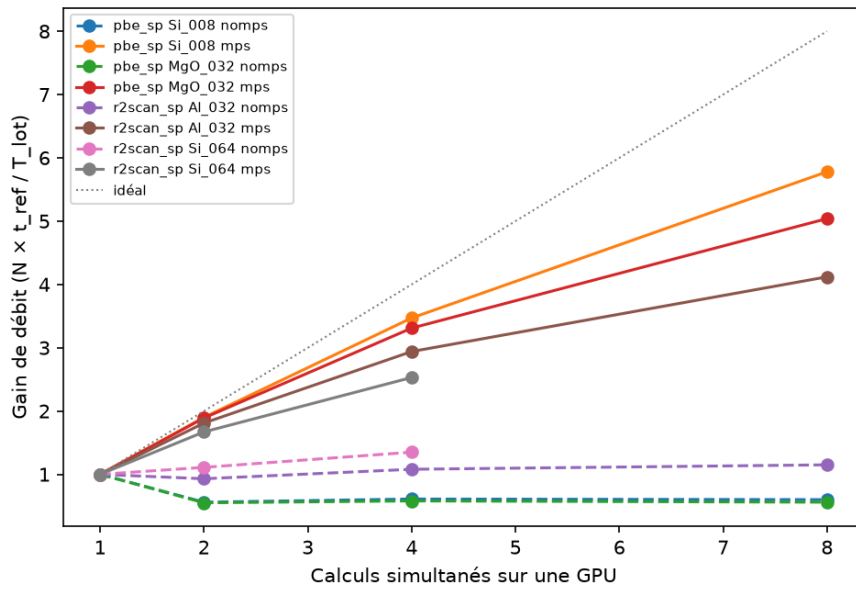


FIGURE 4 – Gain de débit selon le nombre de calculs simultanés sur une GPU (tirets : sans MPS).

- **Avec MPS**, le débit est multiplié par 1,7–1,9 avec 2 calculs, 2,5–3,5 avec 4 et 4,1–5,8 avec 8 ; chaque calcul n'est ralenti que de 1,3 à 1,9 fois à N = 8. L'utilisation moyenne passe de 25–37 % à 69–90 %.

- **Sans MPS**, c'est contre-productif : en PBE, le lot est 1,6 à 1,8 fois plus lent que les mêmes calculs lancés l'un après l'autre. Les changements de contexte coûtent plus que ce que l'on gagne.
- Le gain diminue quand la taille augmente : les plus petits calculs sont ceux qui laissent le plus de place libre.
- La limite pratique n'est pas la mémoire GPU (34,5 Go au plus pour 8 copies) mais la mémoire hôte allouée par Slurm (28 Go par GPU) : Si 64 atomes r²SCAN est limité à 4 copies.
- Les énergies des copies sont identiques à celle du calcul seul.

Mise en œuvre sur vm1-ic2mp. Le démon MPS doit être démarré *dans le job*, avec des répertoires privés, pour ne pas toucher aux autres utilisateurs. Extrait du script `benchmark/run_conc.slurm` :

```
#SBATCH --gres=gpu:1
#SBATCH --ntasks=8
export OMP_NUM_THREADS=1
export CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_${SLURM_JOB_ID}/pipe
export CUDA_MPS_LOG_DIRECTORY=/tmp/mps_${SLURM_JOB_ID}/log
mkdir -p "$CUDA_MPS_PIPE_DIRECTORY" "$CUDA_MPS_LOG_DIRECTORY"
nvidia-cuda-mps-control -d          # démarre le serveur MPS du job

PIDS=()
for c in calc-*; do                # un dossier par calcul
    ( cd "$c" && mpirun -np 1 --bind-to none vasp_std > vasp.stdout 2>&1 ) &
    PIDS+=($!)
done
wait "${PIDS[@]}"                  # attendre ces PID-là, pas un n wait à nu

echo quit | nvidia-cuda-mps-control # arrête le serveur MPS
rm -rf /tmp/mps_${SLURM_JOB_ID}
```

Le `wait` doit porter sur la liste des PID : un `wait` nu attendrait aussi un éventuel moniteur `nvidia-smi` lancé en arrière-plan, et le job ne se terminerait jamais.

6 Ordres de grandeur pour quelques flux de travail

Calcul (1 GPU)	Temps mesuré
Point simple PBE, 1–64 atomes	14 à 163 s
Optimisation r ² SCAN, 64 atomes	12 min (Si) à 48 min (MgO)
Phonons PBE complets (relaxation, supercellules, Born)	0,5 h (Si), 0,8 h (Al), 1,7 h (MgO)
Point simple HSE06, MgO 32 atomes, 12 pas	1,3 h
AIMD PBE+D3, LiN ₃ 144 atomes, <code>vasp_gam</code>	61 s par pas de 1 fs : 200 pas en 3,4 h ; 15 ps ≈ 254 h

Pour l'AIMD, le coût d'une trajectoire de plusieurs picosecondes justifie un champ de force appris à la volée (MLFF de VASP) : la calibration correspondante est en attente de GPU au moment de la rédaction et son gain reste à mesurer.

7 Fiabilité des résultats

- **Reproductibilité des temps** : sur 6 points simples répétés 3 fois, l'écart relatif du temps total ne dépasse pas 1 %. Une différence de plus de quelques pour cent entre deux réglages est donc significative *si* la machine est dans le même état ; d'autres jobs sur le nœud peuvent produire des écarts du même ordre.

- **Énergies** : identiques sur 1 et 2 GPU (0,01 meV/atome), quel que soit KPAR ou NSIM, et pour les calculs simultanés sous MPS.
- **Accord avec Materials Project** : à quelques meV/atome près, sauf deux cas compris (pseudopotentiel Mg plus ancien dans les données PBE de MP, +60 meV/atome constant ; échantillonnage en points k grossier de l'aluminium métallique en supercellule). Ces écarts ne viennent pas de la version GPU.
- **Phonons** : les fréquences obtenues sur 1 et 2 GPU diffèrent d'au plus 5×10^{-4} THz.

8 Recommandations pratiques

Situation	Réglage conseillé
Un calcul jusqu'à ~100 atomes	1 GPU, 1 rang MPI, OMP_NUM_THREADS = 1 ; ne pas toucher à NSIM.
Beaucoup de petits calculs indépendants (criblage, déplacements de phonons, tests de convergence)	4 à 8 calculs par GPU sous MPS, démarré dans le job ; vérifier la mémoire hôte (≤ 28 Go par GPU). Jamais plusieurs calculs sur une GPU sans MPS.
Un calcul urgent, assez gros	2 GPU avec KPAR = 2 si le nombre de points k est pair et ≥ 4 ; KPAR = 1 s'il n'y en a que 3 ou 4 et beaucoup de bandes. Attendre au mieux un gain de 1,5 à 1,9.
Point Γ seul (grosses mailles, AIMD)	vasp_gam : -10 % de temps, -27 % de mémoire.
Hybrides (HSE06)	Le coût ($\sim 60 \times$ PBE par pas) se joue sur les points k et NKRED, pas sur NSIM.
Dynamique moléculaire longue	Envisager un MLFF ; une AIMD pure de 144 atomes coûte ~ 17 h par picoseconde.
Mesures de performance	Noter les autres jobs présents sur le nœud ; répéter au moins une fois si l'écart attendu est inférieur à 5 %.

9 Limites et suite

- **Pas de référence CPU** : seule la version GPU de VASP est installée sur le nœud ; aucune comparaison GPU / CPU n'est possible à partir de ces données.
- **Systèmes de taille moyenne** : l'essentiel des mesures porte sur 1 à 64 atomes ; seuls les scans de NSIM vont jusqu'à 144 et 216 atomes. Le domaine où la GPU est réellement remplie (plusieurs centaines d'atomes) reste à explorer.
- **Une seule exécution par valeur de NSIM**, en partie sur un nœud partagé.
- **AIMD et MLFF** : la calibration vasp_std n'est pas terminée et les calculs MLFF n'ont pas encore tourné.

Suites prévues : production MLFF de LiN_3 (15 ps, 300 à 500 K), puis jeu de diversité de 34 composés MP (points simples puis optimisations), qui élargira les conclusions à des chimies variées et à des mailles jusqu'à 80 atomes.

Données et scripts

Scripts, tableaux et figures sont dans le dépôt

https://github.com/John-Akapulco/GPU_vmp_testing_VASP_etc (dossiers benchmark/ et benchmark/results/). Rapport détaillé en anglais : report/report.tex. Les sorties de VASP restent sur vml-ic2mp.